KLE Technological University
Creating Value Leveraging Knowledge
BVB Campus, Vidyanagar, Hubballi – 580031, Karnataka, INDIA.

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (AI)

Minor Project Report

On

Autonomous QoS-Aware 5G Network Slice Orchestration Using Large Language Models and Multi-Agent Reasoning

submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

COMPUTER SCIENCE AND ENGINEERING (AI)

Submitted By

Simran R Kalkeri	01FE23BCI087
Amaanali D Doddamani	01FE23BCI091
Sparsh Sunil Naik	01FE23BCI085
Rishi P Kulkarni	01FE23BCI088

Under the guidance of

Dr. Narayan D G

School of Computer Science and Engineering (AI)
KLE Technological University, Hubballi

2025–2026

KLE Technological University
Creating Value Leveraging Knowledge
BVB Campus, Vidyanagar, Hubballi – 580031, Karnataka, INDIA.

CERTIFICATE

This is to certify that the project entitled **"Autonomous QoS-Aware 5G Network Slice Orchestration Using Large Language Models and Multi-Agent Reasoning"** is a bonafide work carried out by the student team (Simran R Kalkeri – 01FE23BCI087, Amaanali D Doddamani – 01FE23BCI091, Sparsh Sunil Naik – 01FE23BCI085, Rishi P Kulkarni – 01FE23BCI088), in partial fulfillment of the requirements for the 6th semester B.E. course during the academic year 2025–2026. The project report has been approved as it satisfies the academic requirements prescribed for the said course.

Guide	HoD, CSE(AI)
Dr. Narayan D G	Dr. Narayan D G

External Viva-Voce

Name of the Examiner	Signature with Date
1. _____	_____
2. _____	_____

Acknowledgement

We express our sincere gratitude to our faculty and management for their professional guidance and support throughout the course of this project. We take this opportunity to thank Dr. P.G. Tewari, Vice-Chancellor, Dr. Ashok Shettar, Pro-Chancellor, and Dr. B.S. Anami, Registrar, KLE Technological University, Hubballi, for their vision and support.

We also thank Dr. Narayan D G, HoD, CSE(AI) for the constant guidance during interaction and reviews.

We extend our acknowledgment to the reviewers for critical suggestions and inputs. We also thank Project Co-ordinator Dr. Uday N.Kulkarni and Mr. Sunil V.Gurlahosur for their support during the course of completion.

We express gratitude to our beloved parents for their constant encouragement and support.

Simran R Kalkeri – 01FE23BCI087
Amaanali D Doddamani – 01FE23BCI091
Sparsh Sunil Naik – 01FE23BCI085
Rishi P Kulkarni – 01FE23BCI088

ABSTRACT

Network slicing is a key capability of 5G networks that enables heterogeneous services such as enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC), and massive Machine-Type Communication (mMTC) to coexist on shared infrastructure while maintaining distinct Quality of Service (QoS) requirements. Conventional static and rule-based orchestration approaches lack the ability to reason about the underlying causes of QoS degradation and adapt decisions based on prior operational experience. This work presents an Autonomous QoS-Aware 5G Network Slice Orchestrator that employs a Large Language Model (LLM)-driven multi-agent architecture to perform intelligent slice-level orchestration in a closed-loop control framework. The system is implemented using an Open5GS 5G Standalone core, UERANSIM-based RAN emulation, Kubernetes-managed user-plane functions, and Linux traffic control (tc) mechanisms for dynamic bandwidth enforcement. A LangGraph-based architecture comprising Monitoring, Planning, Validation, Execution, and Memory agents enables an observe–reason–validate–act–learn workflow. The proposed framework introduces structured Chain-of-Thought reasoning for root-cause assessment, a Wrong-Lever Avoidance mechanism based on a relative eMBB load metric, and a Memory-Assisted Decision subsystem that incorporates historical outcomes into future actions. Experimental evaluation across multiple traffic scenarios demonstrates significant improvements over a rule-based baseline, achieving 96.3% URLLC SLA compliance, reducing average recovery time from 8.4 s to 4.2 s, and decreasing unnecessary eMBB throughput degradation by 72%. The results show that LLM-assisted multi-agent orchestration can provide effective, explainable, and adaptive QoS management for next-generation 5G network slicing environments.

Keywords: 5G Network Slicing, Autonomous Orchestration, Large Language Model, Chain-of-Thought Reasoning, Quality of Service, LangGraph, Multi-Agent System, Open5GS, Kubernetes, URLLC, eMBB, mMTC

Contents

List of Tables

List of Figures

Chapter 1 INTRODUCTION

Fifth Generation (5G) mobile networks introduce network slicing as a defining architectural capability, enabling multiple logical network instances—each with distinct performance characteristics—to coexist on shared physical infrastructure. Network slicing permits concurrent support of enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC), and massive Machine-Type Communication (mMTC) services, each with fundamentally different QoS requirements. eMBB demands high throughput and tolerates moderate latency; URLLC requires end-to-end latencies below 20 ms with very high reliability; and mMTC targets energy-efficient transmission for large-scale IoT device populations.

Realising these guarantees in practice is challenging. At the user plane, traffic from different slices may contend for shared buffering and forwarding resources during congestion. Static resource allocations fail to track dynamic traffic demands, while reactive threshold-based controllers lack the ability to reason about the root cause of degradation or to anticipate violations before they occur.

Autonomous orchestration, enabled by Large Language Models (LLMs) and multi-agent frameworks, offers a qualitatively different control paradigm. Rather than pattern-matching against hardcoded thresholds, an LLM-based planning agent performs causal reasoning: identifying which network entity is responsible for congestion, validating whether the proposed control action addresses that cause, and consulting prior operational history stored in an agent memory to refine its decisions. This project implements, deploys, and evaluates such a system in a cloud-native 5G testbed.

1.1 Motivation

The evolution from rule-based to autonomous orchestration is driven by three limitations observed in practical 5G slice management systems.

First, rule-based controllers are structurally blind to causation. A threshold-based system that detects elevated URLLC RTT will throttle eMBB bandwidth regardless of whether eMBB traffic is actually active. When eMBB load is low, this action is unnecessary and harmful. An agent capable of reasoning about relative load—quantified by the `embb_load_fraction` metric introduced in this work—can correctly distinguish idle from bursting eMBB and decline to act when throttling is ineffective.

Second, reactive control cannot prevent violations; it can only respond to them. A URLLC SLA breach lasting even 200 ms may disrupt a latency-critical industrial automation session. Pre-emptive action, triggered by trend analysis and predictive causal reasoning, is only possible when the controller possesses a causal model of the network state rather than a simple threshold table.

Third, accumulated operational experience is wasted in memoryless systems. An agent

that stores the outcome of prior actions—including whether throttling eMBB at a given load level subsequently resolved the RTT violation—can converge to more accurate control policies over time without requiring offline training.

1.2 Literature Survey

Bandara et al. propose an AI-native framework for 6G network slice orchestration, monitoring, and trading using multiple autonomous AI agents and Large Language Models (LLMs) [11]. The architecture consists of layered components that perform intent interpretation, slice deployment, SLA monitoring, policy enforcement, and resource allocation. The framework integrates the Model Context Protocol (MCP) to translate natural language requests into secure orchestration actions. A real-world testbed based on Open5GS, Ericsson RAN, and Kubernetes was used for evaluation. Experimental results showed successful LLM fine-tuning, accurate Kubernetes manifest generation, reduced hallucinations, and reliable MCP tool invocation. The study demonstrates that agentic AI can improve automation, scalability, and intelligent management of future 6G network slicing environments.

Grings et al. propose a full dynamic orchestration framework for 5G core network slicing over a cloud-native Kubernetes platform [12]. Unlike existing approaches that only support partial orchestration through resource scaling, the proposed solution introduces a Kubernetes-integrated controller capable of dynamically reconfiguring 5G core functions, including NSSF, AMF, SMF, and UPF, during runtime without terminating active network slices. The framework integrates cloud orchestration with 5G core management using REST APIs and Kubernetes networking components to enable seamless slice adaptation and resource allocation. Experimental evaluation using a free5GC-based testbed showed that network slices could be reconfigured without service interruption, while reducing slice reconfiguration requests by approximately 47.5% compared to partial orchestration approaches. The results demonstrate improved service continuity, lower management overhead, and enhanced adaptability for dynamic 5G network slicing environments.

Novanana et al. propose a Kubernetes-based MANO framework for provisioning coexisting eMBB and URLLC services through 5G network slicing using open-source technologies [13]. The architecture integrates free5GC, UERANSIM, OpenStack, and kube5gnfvo to create multiple network slices on a shared physical infrastructure. The authors evaluate two slicing scenarios: one using separate UPFs for each slice and another using dedicated SMF and UPF pairs per slice to achieve greater isolation and service customization. Experimental results show that the second approach delivers better performance, achieving more stable throughput of 20.2–22.1 Gbps, fewer retransmissions, and improved congestion control compared to the first scenario. The study demonstrates the feasibility of deploying flexible and scalable 5G network slices for diverse service requirements using open-source cloud-native tools, although the evaluation is limited to a virtualized test environment and does not consider large-scale dynamic traffic conditions.

Reddy proposes a Deep Reinforcement Learning (DRL)-based Kubernetes autoscaling framework for managing high-throughput 5G network functions and network slices [14]. The approach utilizes a Deep Q-Learning (DQN) agent to dynamically allocate re-

sources and scale Kubernetes pods based on real-time network conditions, overcoming the limitations of traditional threshold-based autoscaling mechanisms. The proposed system is designed to support multiple concurrent network slices while maintaining strict QoS requirements and efficient resource utilization. Experimental results show significant improvements, including 71.7% faster slice deployment, 35% higher resource utilization, 22% lower latency, and 40% faster processing times compared to conventional methods. The framework also achieved up to 96.7% spectrum efficiency and reduced service-level agreement violations while maintaining reliable performance across dynamic 5G workloads. The study demonstrates the potential of AI-driven autoscaling for future cloud-native 5G and beyond-5G networks, although the evaluation is limited to a simulated test environment.

Dandoush et al. propose an LLM-powered network slicing management and orchestration framework that integrates Large Language Models (LLMs) and multi-agent systems with existing ETSI MANO and 3GPP slicing architectures [15]. The framework enables LLM agents to translate user intents expressed in natural language into technical slice requirements, automate slice design and deployment, coordinate resources across multiple administrative domains, and continuously monitor network performance. Multiple specialized agents collaborate to perform slice modeling, resource allocation, lifecycle management, and SLA assurance while simplifying interactions for both network operators and service consumers. The authors demonstrate how LLM-assisted orchestration can improve automation, interoperability, and dynamic resource management in complex multi-domain environments. Although the work presents a detailed architectural vision and use cases, it remains a conceptual framework and does not provide experimental validation or quantitative performance results.

Sulaiman et al. propose MicroOpt, a model-driven framework for dynamic resource optimization in 5G network slicing that uses a differentiable deep neural network (DNN) model and gradient-based optimization to allocate resources while satisfying QoS constraints [16]. The framework combines traffic prediction, QoS modeling, and a primal-dual optimization algorithm to dynamically scale CPU and bandwidth resources according to changing network conditions. It was implemented on an open-source 5G testbed using srsRAN, Open5GS, Kubernetes, ONOS, Prometheus, and Grafana. Experimental results show that MicroOpt achieves up to 21.9% lower resource consumption than state-of-the-art approaches such as Atlas and CaDRL, while maintaining QoS degradation within SLA limits. The study demonstrates that differentiable AI-based optimization can provide more efficient and adaptable resource management for 5G and beyond network slicing environments.

Saha et al. propose MonArch, a scalable monitoring architecture for cloud-native 5G deployments that focuses on network slice monitoring and KPI computation [17]. The framework integrates components such as Prometheus, Elasticsearch, Apache Kafka, Logstash, and Kubernetes to collect, correlate, and analyze monitoring data across different network segments. MonArch introduces a northbound API that allows applications to specify slice-level monitoring requests and automatically compute KPIs such as slice throughput. The architecture was validated on a Kubernetes-based 5G testbed using Free5GC and UERANSIM, where multiple network slices were monitored in real time. Experimental results show that MonArch scales efficiently with the number of slices, introduces minimal data-ingestion overhead, and achieves an effective balance between monitoring accuracy and resource consumption using a 5-second monitoring

interval. The study demonstrates the importance of scalable monitoring frameworks for enabling AI-driven orchestration and closed-loop management in future 5G networks.

Tran et al. propose PCLANSA, a machine-learning-based closed-loop service assurance framework for 5G and B5G network slicing that dynamically allocates and scales resources across end-to-end network slices [18]. The framework uses LSTM-based traffic prediction to forecast future demand and automatically perform VNF scaling and link-capacity adjustments before QoS degradation occurs. It was evaluated in an OMNeT++-based 5G slicing environment supporting eMBB, uRLLC, mMTC, and VoIP services. Experimental results showed that PCLANSA effectively maintains QoS by reducing KPI violations related to delay, jitter, and packet loss while significantly improving resource utilization. Compared to static provisioning, the framework achieved resource savings of 54.85% for eMBB, 57.1% for uRLLC, 50.87% for mMTC, and 23.63% for VoIP slices, demonstrating the effectiveness of AI-driven closed-loop resource management for future 5G/B5G networks.

1.3 Problem Statement

To design and evaluate an autonomous agent-based orchestration framework for 5G network slicing that employs LLM-assisted causal reasoning and dynamic QoS control to protect slice performance and reduce unnecessary control actions compared to a rule-based baseline.

1.4 Objectives and Scope of the Project

1.4.1 Objectives

1. To design and develop a cloud-native 5G network slicing testbed integrating Open5GS, UERANSIM, Kubernetes-hosted per-slice UPFs, and dynamic user-plane QoS enforcement mechanisms for eMBB, URLLC, and mMTC services.
2. To implement an autonomous LLM-assisted multi-agent orchestration framework incorporating monitoring, planning, execution, memory, and structured causal reasoning capabilities for intelligent network slicing management and control.
3. To evaluate the effectiveness of the proposed framework against a rule-based baseline through behavioral audits and QoS performance analysis using metrics such as URLLC latency compliance, eMBB throughput preservation, mMTC packet delivery ratio, and recovery time.

1.4.2 Scope of the Project

The project focuses on a single-domain 5G slice orchestration system operating at the user-plane level through dynamic bandwidth shaping. The orchestration intelligence is provided by a locally hosted LLM to preserve data sovereignty and eliminate cloud API dependencies. Extensions to multi-domain slicing, RL-based policy refinement, and O-RAN xApp integration are identified as directions for future work.

Chapter 2 REQUIREMENT ANALYSIS

2.1 Functional Requirements

- The system shall support the simultaneous operation of eMBB, URLLC, and mMTC network slices on shared 5G infrastructure with user-plane isolation.
- The system shall establish multiple concurrent PDU sessions for a single UE across all configured network slices.
- The system shall enforce slice-specific bandwidth allocation and QoS policies using Linux traffic control mechanisms.
- The system shall continuously monitor per-slice QoS metrics (throughput, latency, and packet loss) for network state assessment.
- The system shall dynamically adjust slice bandwidth allocations based on observed QoS conditions through a closed-loop controller.
- The system shall support predictive resource management by forecasting QoS degradation and proactively reallocating resources.
- The system shall log monitoring data and control actions for analysis, visualization, and future optimization.

2.2 Non-Functional Requirements

- The monitoring thread shall operate at a 3-second interval, independent of LLM inference latency.
- The Planning Agent shall enforce a maximum inference timeout of 60 s, after which a rule-based fallback shall be invoked.
- `tc` shaping commands shall be applied with near-real-time responsiveness following action validation.
- The Agent Memory shall retain the 10 most recent action-outcome episodes.
- The system shall support experimental evaluation through comprehensive decision logging (reasoning outputs, confidence scores, and execution outcomes) and an optional dry-run mode that prevents the application of `tc` commands.

2.3 Hardware Requirements

Table 2.1: Hardware Configuration of the Testbed

Node	**Role**	**RAM**	**Storage**
kubemaster (192.168.49.174)	K8s Control Plane / Orchestrator	16 GB	256 GB
kube (192.168.49.173)	K8s Worker 1 / Primary UPF pods	16 GB	256 GB
kube2 (192.168.49.181)	K8s Worker 2 / Replica UPF pods	16 GB	256 GB
core VM (192.168.49.143)	Open5GS 5GC (AMF, SMF, NRF ...)	8 GB	128 GB
UERANSIM VM (192.168.49.139)	gNB + UE Emulation	8 GB	128 GB
Total		**64 GB**	**1 TB**

2.4 Software Requirements

Table 2.2: Software Stack and Version Requirements

Component	**Technology**	**Version**
Operating System	Ubuntu 22.04 LTS	All nodes
Container Orchestrator	Kubernetes	v1.29.15
Container Runtime	containerd	v1.7.28
CNI Plugin	Flannel (VXLAN)	v0.22
5G Core	Open5GS	v2.7.x
RAN / UE Emulation	UERANSIM	v3.2.7
LLM Backend	Ollama	v0.3.x
LLM Model	llama3.2:3b	3B param, quantised
Agent Framework	LangGraph	v0.1.x
Monitoring	Prometheus	v2.45
Visualisation	Grafana	v10.x
Traffic Control	Linux `tc` / HTB	Kernel 6.8
Programming Languages	Python 3.10, Bash, YAML	—

Chapter 3 SYSTEM DESIGN

3.1 End-to-End Orchestrator Architecture

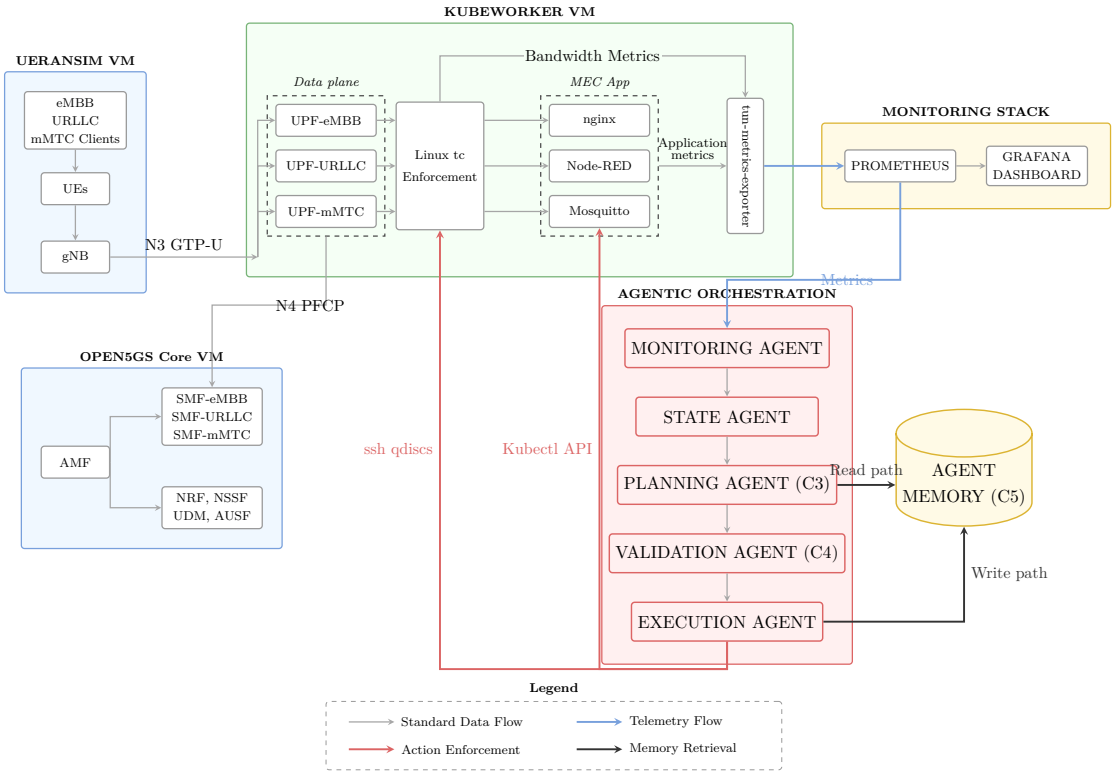

Figure 3.1: System Architecture of the Proposed Agentic 5G Slice Orchestrator.

Figure 3.1 presents the overall architecture of the proposed Agentic 5G Slice Orchestrator. The infrastructure consists of a UERANSIM-based radio access network, an Open5GS standalone 5G core, and a Kubernetes worker node hosting dedicated UPFs and MEC applications for the eMBB, URLLC, and mMTC slices. User traffic generated by the UEs traverses the gNB, Open5GS core, and slice-specific UPFs before reaching the MEC applications. QoS isolation is enforced at the user plane through Linux tc traffic shaping, which dynamically adjusts eMBB bandwidth allocation while preserving the performance requirements of latency-sensitive URLLC traffic. In addition to traffic shaping, the architecture supports cloud-native resource adaptation through Kubernetes-based MEC application scaling.

A monitoring stack comprising a custom metrics exporter, Prometheus, and Grafana continuously collects network and application telemetry from the Kubernetes environment. These metrics are consumed by the Agentic Orchestration layer, which implements a LangGraph-based multi-agent workflow consisting of Monitoring, State, Planning, Validation, and Execution agents. The Planning Agent performs structured

Chain-of-Thought reasoning (C3) using current network conditions and historical context retrieved from the **Agent Memory (C5)**. Candidate actions are subsequently verified by the **Validation Agent (C4)** before being executed through Linux tc commands for bandwidth enforcement or Kubernetes API (kubectl) operations for MEC application scaling. The resulting action outcomes are written back to memory, forming a closed-loop MAPE-K control cycle that enables adaptive, explainable, and memory-assisted network slice orchestration.

3.2 Agentic Orchestration Framework

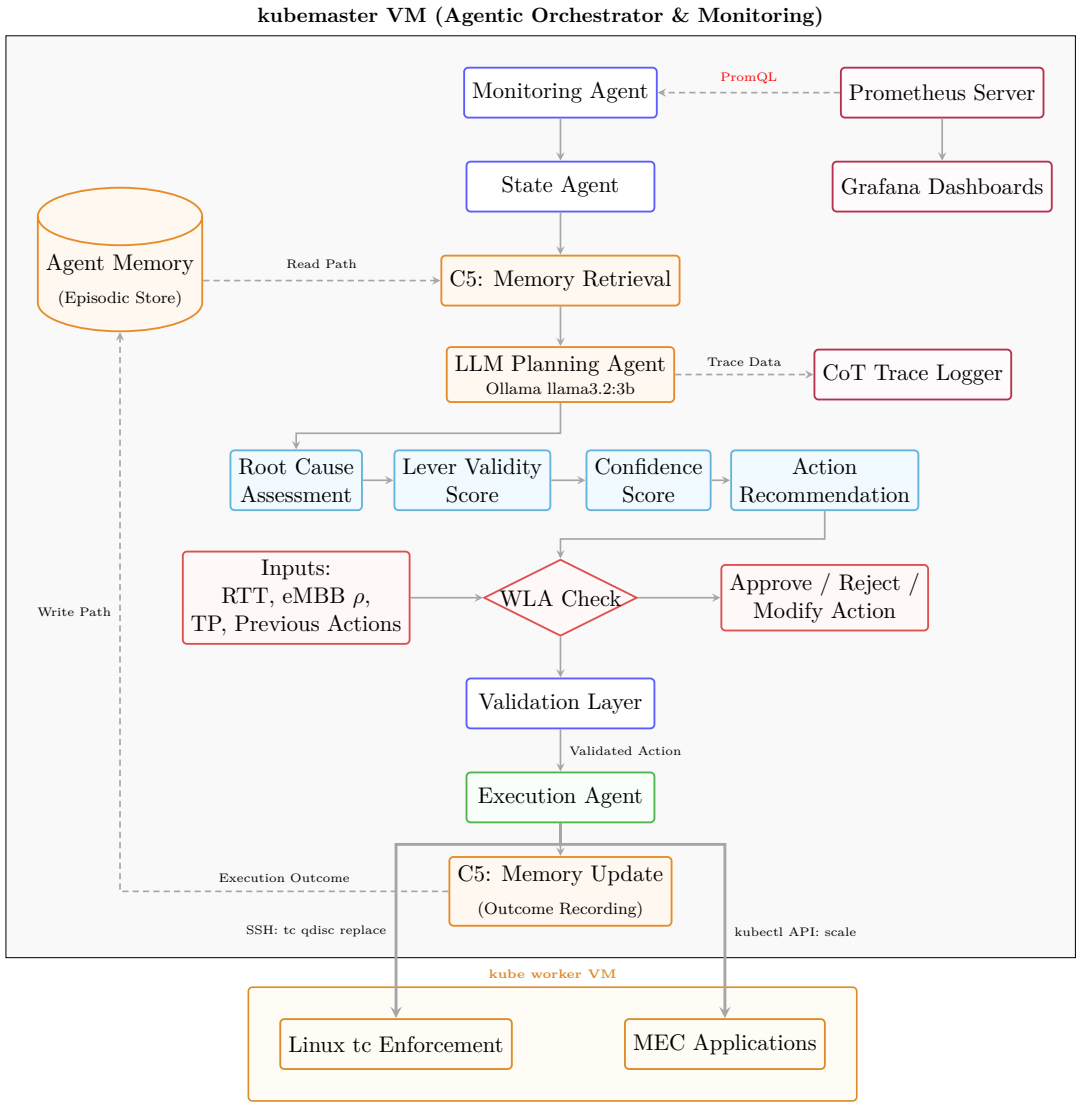

Figure 3.2: Agentic Orchestration Framework.

As shown in Figure 3.2, the proposed agentic orchestration framework is implemented as a LangGraph-based multi-agent system deployed on the Kubernetes master node. Telemetry collected from Prometheus is processed by the Monitoring and State Agents, while historical actionoutcome records are retrieved from the episodic memory store to provide contextual knowledge. The LLM Planning Agent performs Chain-of-Thought

reasoning to generate root-cause assessments, lever-validity scores, confidence estimates, and recommended actions. These decisions are subsequently examined by the Wrong-Lever Avoidance (WLA) and Validation layers to prevent unsafe or ineffective control actions. Validated actions are executed through the Execution Agent, which applies bandwidth shaping via Linux tc and controls MEC applications through Kubernetes APIs. The resulting network outcomes are recorded back into memory, creating a continuous feedback loop that enables adaptive, explainable, and memory-assisted orchestration.

3.3 Orchestration Decision Lifecycle

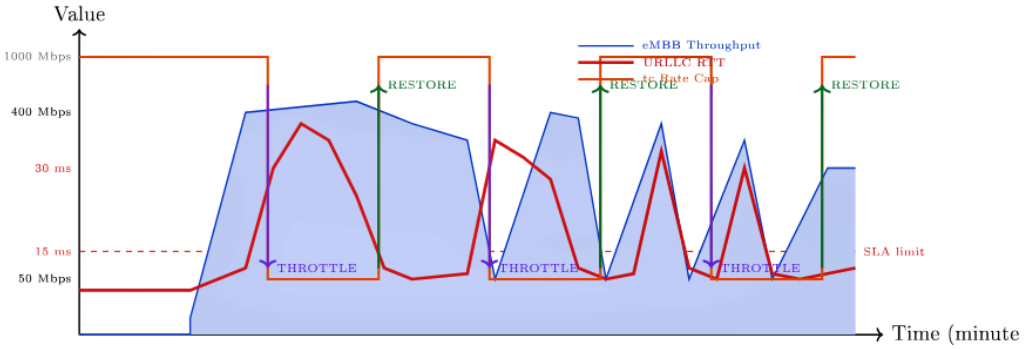

Figure 3.3: Contention and recovery lifecycle demonstrating the sequence of eMBB surge, SLA breach, autonomous throttling, and subsequent restoration.

Figure 3.3 visualises the autonomous SLA recovery cycle observed live in the testbed environment. The orchestration decision lifecycle operates as a continuous closed loop: a sudden surge in eMBB traffic leads to cross-slice contention, causing a breach of the latency-sensitive URLLC SLA threshold. In response, the agentic framework detects the degradation, evaluates the root cause, and issues an autonomous throttle command to temporarily restrict eMBB throughput. This targeted bandwidth shaping alleviates the contention, allowing the URLLC Round-Trip Time (RTT) to recover rapidly below the critical SLA limit. Once stable conditions are restored, the orchestrator safely lifts the rate cap via a restore event, maximising resource utilization until the cycle naturally repeats.

3.4 Mathematical Foundations

3.4.1 eMBB Load Fraction

The eMBB relative load fraction is the central metric enabling Contribution C4. It calculates the current utilization of the eMBB slice relative to its recent historical peak. Unlike absolute throughput, it is invariant to traffic profile differences, allowing the orchestrator to accurately identify genuine congestion sources regardless of baseline traffic volume.

$$\rho_{\text{eMBB}}(t) = \frac{\lambda(t)}{\max\limits_{t' \in [t - W\Delta, t]} \lambda(t')} \tag{3.1}$$

where,

- $\rho_{\text{eMBB}}(t)$: eMBB relative load fraction at time t
- $\lambda(t)$: eMBB packet rate at time t (pkt/s)
- W : Rolling window size (cycles)
- Δ : Monitoring interval (s)

A value $\rho_{\text{eMBB}} < 0.2$ indicates a low-activity slice that should not be throttled regardless of RTT. Note that $\rho_{\text{eMBB}} \in [0, 1]$.

3.4.2 SLA Compliance Ratio

This equation calculates the percentage of total monitoring cycles where the URLLC RTT strictly complies with the defined latency threshold. It serves as the primary success metric for evaluating the orchestrator's ability to protect critical latency-sensitive services during periods of heavy cross-slice congestion.

$$C_{\text{sla}} = 1 - \frac{N_V}{N_T} = \frac{\#\{t : R(t) \leq \theta\}}{N_T} \tag{3.2}$$

where,

- C_{sla} : SLA compliance ratio
- $R(t)$: URLLC RTT_{99} at time t (ms)
- θ : URLLC SLA threshold (ms)
- N_T : Total monitoring cycles (cycles)
- N_V : Cycles with SLA violation ($R(t) > \theta$)

3.4.3 Recovery Time

Recovery time quantifies the time interval required for the orchestrator to successfully restore network stability following a latency violation. It plays a crucial role in evaluating how the Memory-Assisted Decision mechanism (C5) accelerates the system's return to a normal operational state.

$$T_{\text{rec}} = t_{\text{clear}} - t_{\text{breach}}, \quad t_{\text{clear}} = \min\{t > t_{\text{breach}} : R(t) \leq \theta\} \tag{3.3}$$

where,

- T_{rec} : SLA recovery time (s)
- t_{breach} : Timestamp of the initial SLA violation (s)
- t_{clear} : Timestamp when latency is successfully restored below threshold (s)

3.4.4 Throughput Preservation Ratio

The throughput preservation ratio measures the proportion of maximum allowable eMBB bandwidth that is successfully maintained over the course of an experiment. This metric is essential for proving the effectiveness of the Wrong-Lever Avoidance mechanism (C4) in minimizing unnecessary bandwidth throttling when eMBB is not the root cause of congestion.

$$\mathrm{TP} = 1 - \frac{\sum_{t:r(t)<r_{\max}} (r_{\max} - r(t)) \cdot \Delta}{r_{\max} \cdot N_T \cdot \Delta} \tag{3.4}$$

where,

- TP : Throughput preservation ratio
- $r(t)$: eMBB `tc` rate cap applied at time t (Mbit/s)
- $r_{\max}$: Maximum provisioned eMBB rate (Mbit/s)
- Δ : Monitoring interval (s)
- N_T : Total monitoring cycles (cycles)

3.4.5 Action Efficiency

Action efficiency calculates the precision of the orchestrator's control decisions by identifying the fraction of throttle actions that correctly responded to a genuine SLA violation or a strictly rising latency trend. A high efficiency score demonstrates that the multi-agent system successfully avoids spurious, reactive, or incorrect network control actions.

$$\eta_{\mathrm{act}} = \frac{\#\{\text{throttle} : R > \theta \text{ or trend rising}\}}{\#\{\text{throttle actions}\}} \tag{3.5}$$

where,

- η_{act} : Action efficiency ratio
- R : Monitored URLLC RTT_{99} (ms)
- θ : URLLC SLA threshold (ms)

Chapter 4 ALGORITHM DESIGN

This chapter presents the six core algorithms that constitute the Agentic Orchestrator. Each algorithm is formally defined with pseudocode, input/output specifications, and complexity analysis.

4.1 Algorithm 1: Monitoring and State Collection

Algorithm 1 Monitoring Agent: Decoupled State Collection

Input: Monitoring interval $\Delta = 3\,\text{s}$, SSH credentials, Prometheus endpoint
Output: Updated thread-safe state cache $\mathcal{S}$

while orchestrator active **do**
  $t_0 \leftarrow \text{time}()$
  // **Channel 1: URLLC Telemetry via SSH logs**
  $R \leftarrow \text{SSH.exec}(\texttt{"tail -1 /tmp/mec-clients/urllc-*.log"})$
  $R \leftarrow \text{parseRTT}(R)$ ▷ extract RTT_{99} in ms
  $d, F_{\text{urllc}} \leftarrow \text{countDeadAndFailedTunnels}()$ ▷ dead tunnels and URLLC drop count
  // **Channel 2: eMBB Telemetry via Prometheus (UPF tun metrics)**
  $\lambda_{\text{bps}} \leftarrow \text{PromQL}($ `"irate(tun_tx_bytes{interface='ogstun-embb'}[30s]) * 8"` $)$
  $\lambda_{\text{pkt}} \leftarrow \text{PromQL}($ `"irate(tun_tx_packets{interface='ogstun-embb'}[30s])"` $)$
  $C \leftarrow 1000 \times \text{PromQL}($ `"rate(container_cpu_usage_seconds_total{namespace='embb'}[30s])"` $)$
  // **Channel 3: mMTC Delivery via MQTT Logs (SSH)**
  $N_{\text{pub}}, N_{\text{ok}} \leftarrow \text{parseMQTTLog}()$
  $\text{PDR} \leftarrow N_{\text{ok}}/N_{\text{pub}}$ **if** $N_{\text{pub}} > 0$ **else** 1.0
  // **Channel 4: Enforcement State via SSH**
  $r_{\text{tc}} \leftarrow \text{SSH.exec}(\texttt{"tc qdisc show dev ogstun-embb"})$
  $r_{\text{tc}} \leftarrow \text{parseTCRate}(r_{\text{tc}})$ ▷ current shaping rate in Mbit/s
  // **Derived metrics & Contribution C4 (Wrong-Lever Avoidance)**
  $\rho_{\text{eMBB}} \leftarrow \lambda_{\text{pkt}}/\lambda_{\max}$ ▷ Algorithm **??**
  $\text{tc}_{\text{state}} \leftarrow$ **if** $r_{\text{tc}} < r_{\max}$ **then** `THROTTLED` **else** `NORMAL`
  // **Thread-safe state cache write**
  $\mathcal{S}$_lock.acquire()
  $\mathcal{S} \leftarrow \left\{R,\ \lambda_{\text{bps}},\ \lambda_{\text{pkt}},\ \rho_{\text{eMBB}},\ \text{PDR},\ d,\ F_{\text{urllc}},\ C,\ r_{\text{tc}},\ \text{tc}_{\text{state}}\right\}$
  $\mathcal{S}$_lock.release()
  $\text{sleep}(\max(0, \Delta - (\text{time}() - t_0)))$
end while

This algorithm continuously polls network metrics to maintain an up-to-date representation of the user plane. It operates on a strict 3-second interval, ensuring state freshness independent of LLM execution times.

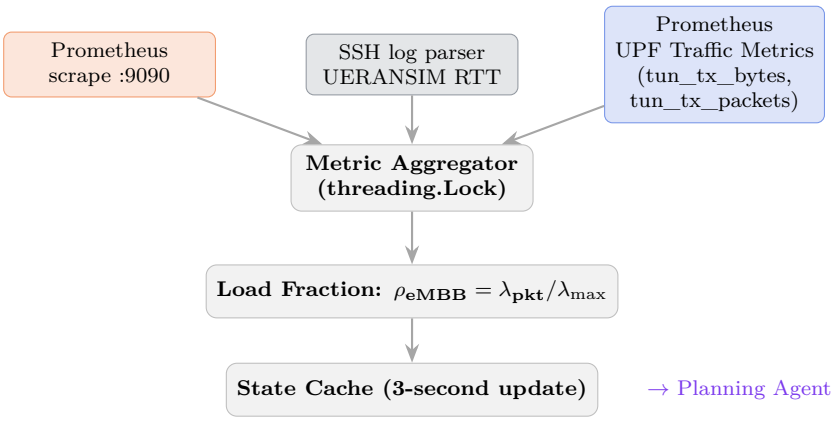

Figure 4.1: Monitoring Agent Flowchart.

The Monitoring Agent executes as an independent thread with a fixed collection interval of $\Delta = 3\,\text{s}$. By decoupling telemetry collection from LLM inference, the orchestrator maintains a consistent state refresh rate regardless of planning latency. The resulting state cache $\mathcal{S}$ serves as the primary input to the State Agent and subsequent Chain-of-Thought reasoning process. The $O(1)$ complexity ensures that telemetry collection safely completes before the next cycle begins.

4.2 Algorithm 2: Chain-of-Thought Planning and Action Generation (C3)

Algorithm 2 Chain-of-Thought Planning and Action Generation (C3)

Input: State $\mathcal{S}$, Agent Memory $\mathcal{M}$ (last $K = 10$ entries), System Prompt π
Output: Action Plan $\mathcal{A} = \{$
action, reasoning, confidence, root_cause_assessment, new_rate_int,
contradiction, lever_validity$\}$

// 1. Structured Prompt Construction
$\mu \leftarrow$ summariseMemory($\mathcal{M}$) ▷ retrieve last K causal episodes
$\sigma \leftarrow$ formatStateVector($\mathcal{S}$) ▷ encode telemetry and SLA status
prompt $\leftarrow \pi \oplus \sigma \oplus \mu$
// 2. LLM Inference
raw_json $\leftarrow$ LLM.generate(prompt, `format=json`, `timeout` $= 60$)
// 3. Failure Handling & Schema Validation
if raw_json $= \emptyset$ **or** isMalformed(raw_json) **then return** RuleBasedFallback($\mathcal{S}$)
▷ fallback on timeout or JSON parse error
end if
cot $\leftarrow$ parseJSON(raw_json)
// 4. Chain-of-Thought Parsing
action $\leftarrow$ cot.action
confidence $\leftarrow$ float(cot.confidence)
root_cause $\leftarrow$ cot.root_cause_assessment
lever_validity $\leftarrow$ cot.lever_validity
// 5. Chain-of-Thought Consistency Evaluation
$\mathcal{K}_{\text{deny}} \leftarrow \{$*"not embb", "embb is not", "embb not", "low load", "nearly idle", "not the cause", "cannot help", "wrong lever", "unlikely", "idle"*$\}$
contradiction $\leftarrow$ False
if action $=$ `throttle_embb` **and** $\exists k \in \mathcal{K}_{\text{deny}} : k \in$ lower(root_cause) **then**
  contradiction $\leftarrow$ True
  confidence $\leftarrow \min$(confidence, 0.35)
  log.warning(`"CONTRADICTION: root_cause denies eMBB congestion"`)
end if
// 6. Action Plan Generation
$\mathcal{A} \leftarrow \Big\{$
  action: action,
  reasoning: cot.reasoning,
  confidence: confidence,
  root_cause_assessment: root_cause,
  lever_validity: lever_validity,
  contradiction: contradiction,
  new_rate_int: int(cot.new_rate_int) **or** $r_{\max}$
$\Big\}$
return $\mathcal{A}$

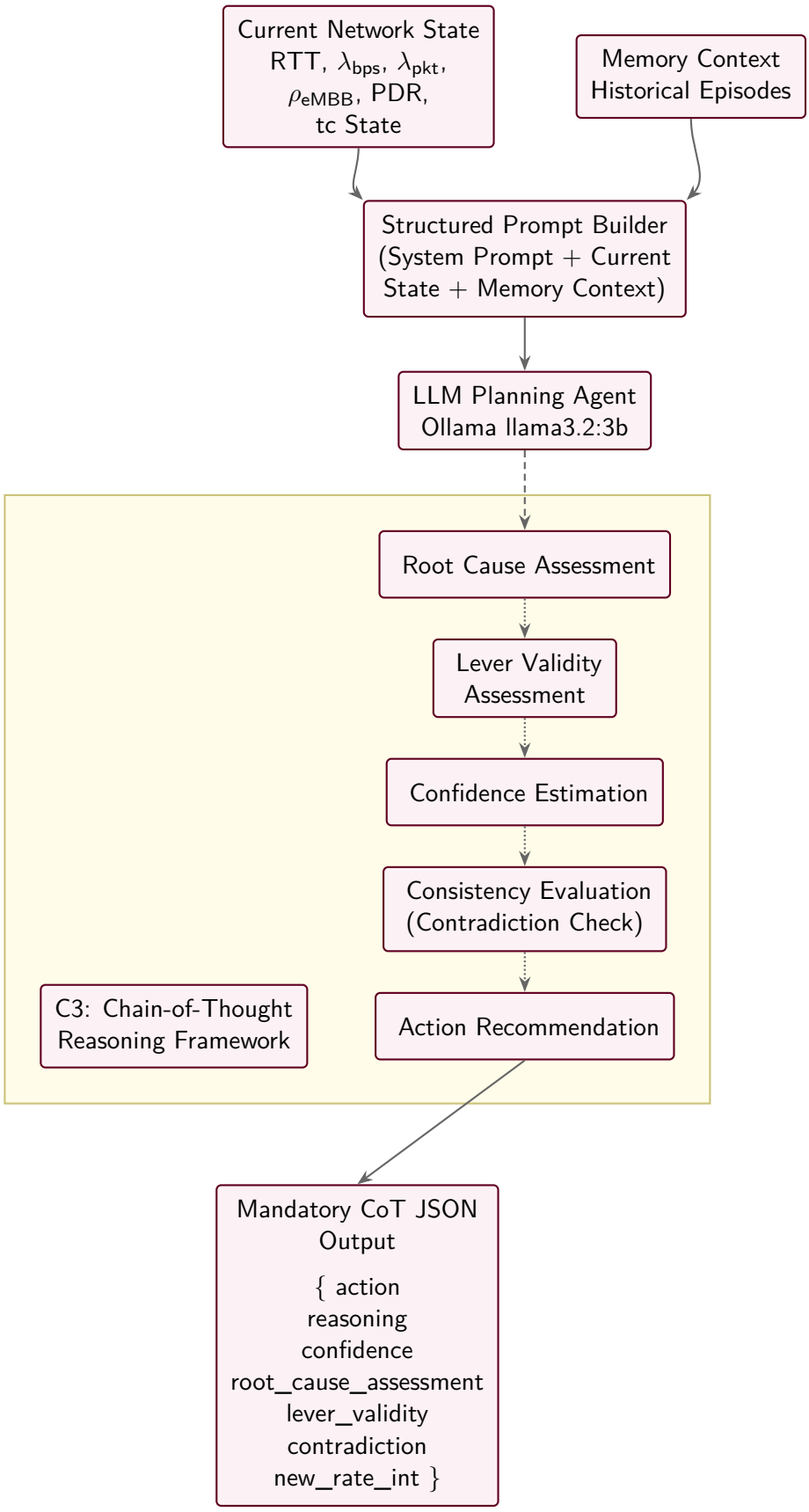

Figure 4.2: Planning Agent Flowchart.

The Planning Agent acts as the cognitive core of the orchestrator. By mandating a structured Chain-of-Thought (CoT) schema (Contribution C3), the algorithm explicitly forces the LLM to deduce the network's root cause before making a decision. Crucially, the Consistency Evaluation stage parses this reasoning. If the model determines eMBB is *not* the root cause, yet still selects a throttle action, the algorithm detects this logic fault and severely caps the confidence, ensuring the system intercepts flawed logic before it translates into a detrimental network configuration. The overall runtime is strictly dominated by the LLM inference latency.

4.3 Algorithm 3: Wrong-Lever Avoidance Validation (C4)

Algorithm 3 Wrong-Lever Avoidance Validation (C4)

Input: CoT output $\{a, r', c, \text{contradiction}, \text{lever\_validity}\}$, bounds $[r_{\min}, r_{\max}]$, current tc rate r_{current}, State $\mathcal{S}$

Output: Safe action a^*, target rate r^*, adjusted confidence c^*, lever validity lever_validity

// 1. Dead-tunnel guard
if $\mathcal{S}$.dead_tunnels $= 3$ **then** ▷ all URLLC tunnels dead **return** {`no_action`, $r_{\max}$, 0.0, lever_validity} ▷ monitoring unreliable
end if
// 2. Range clamp
$r^* \leftarrow \max(r_{\min}, \min(r', r_{\max}))$
// 3. Confidence adjustment based on contradiction
if contradiction = True **then**
    $c^* \leftarrow \min(c, 0.35)$ ▷ penalize contradictory reasoning
    log.warning(`"Applying contradiction penalty"`)
else
    $c^* \leftarrow c$
end if
// 4. Redundant-action guard
if a = `throttle_embb` **and** $r^* = r_{\text{current}}$ **then return** {`no_action`, r_{current}, c^*, lever_validity}
end if
return $\{a, r^*, c^*, \text{lever\_validity}\}$

This algorithm acts as a safeguard against ineffective control actions proposed by the Planning Agent. It inspects the LLM's logical consistency and blocks throttling attempts when eMBB is not the true source of congestion.

where,

- r_{current} : Current eMBB shaping rate configured on `ogstun-embb`.

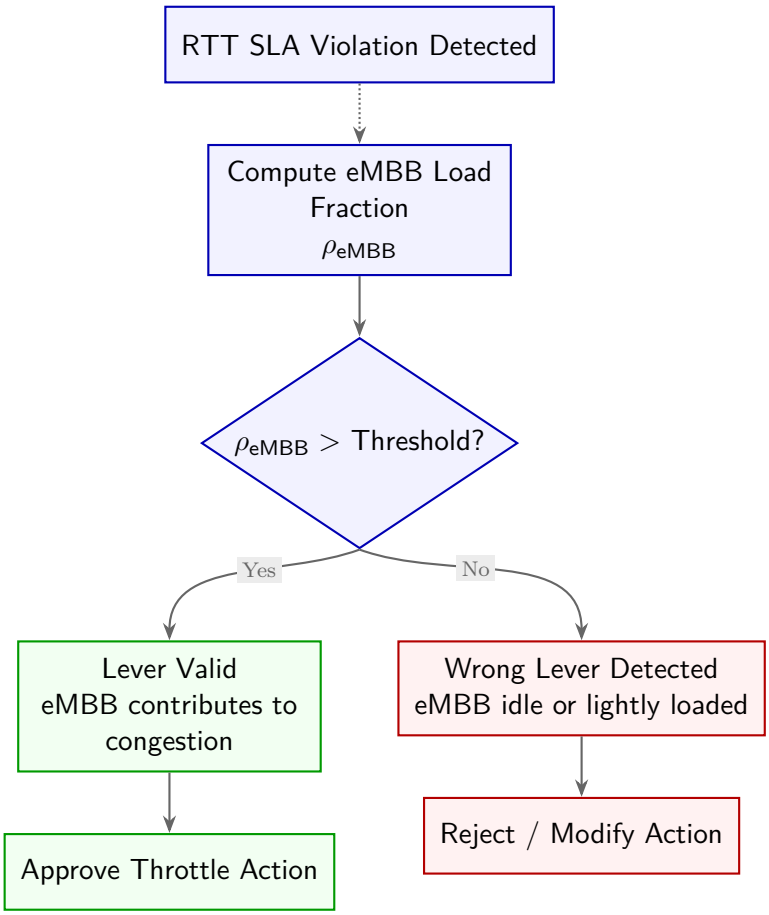

Figure 4.3: Wrong-Lever Avoidance (WLA) Flowchart.

The Wrong-Lever Avoidance mechanism provides essential deterministic grounding to the probabilistic LLM. When an RTT SLA violation occurs, the algorithm checks the parsed root cause against a predefined list of denial keywords alongside the mathematical load fraction constraint. If it detects that the eMBB slice is lightly loaded but the LLM still elected to throttle it, the system marks this as a wrong-lever scenario. It penalizes the confidence score and potentially modifies the action to prevent degrading unrelated traffic streams, achieving reliable fail-safe operation.

4.4 Algorithm 4: Memory-Assisted Decision Process (C5)

Algorithm 4 Memory-Assisted Decision Process (C5)

Input: Action a, pre-action state $\mathcal{S}_{\text{pre}}$, post-action RTT R_{post}, Memory $\mathcal{M}$ (deque, maxlen $K = 10$)
Output: Updated $\mathcal{M}$ with outcome

$\Delta R \leftarrow R_{\text{post}} - \mathcal{S}_{\text{pre}}.\text{RTT}$ ▷ outcome delta
outcome $\leftarrow$ **if** $\Delta R < -2$: `"improved"` **elif** $\Delta R > 2$: `"worsened"` **else**: `"neutral"`
$e \leftarrow \{\text{ts}, a, \mathcal{S}_{\text{pre}}.\rho, \mathcal{S}_{\text{pre}}.\text{RTT}, R_{\text{post}}, \Delta R, \text{outcome}\}$
$\mathcal{M}.\text{append}(e)$ ▷ oldest entry evicted if full
// Context summary for next LLM prompt
summary $\leftarrow \emptyset$
for all $e_i \in \mathcal{M}$ **do**
    summary $\leftarrow$ summary + "`[` $e_i.\text{ts}$ `]` $a_i : \rho = \rho_i, \Delta R = \Delta R_i$ (outcome$_i$)"
end forreturn $\mathcal{M}$, summary

This algorithm manages the short-term episodic memory of the orchestrator. It records the delta between pre-action and post-action latency to classify decisions as improved, worsened, or neutral.

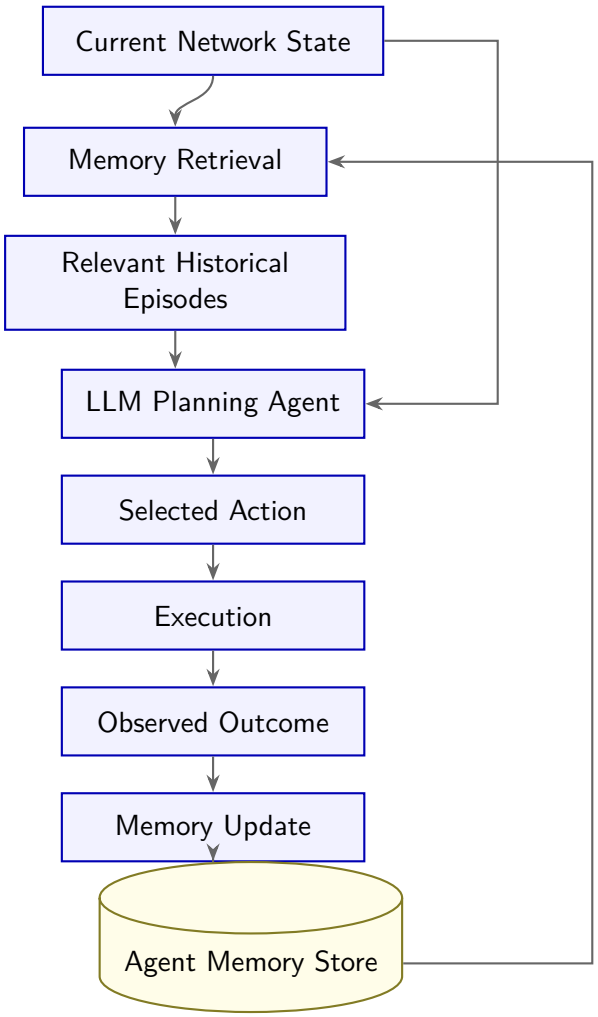

Figure 4.4: Memory-Assisted Decision Lifecycle Flowchart.

The Memory-Assisted Decision Process acts as a feedback loop that evaluates the tangible network impact of previous actions. By maintaining a sliding window queue of size $K = 10$, it captures the immediate outcome of every executed command without bloating the context window. This episodic memory store is continuously appended to the LLM's system prompt during the planning phase. Providing this historical timeline allows the agent to recognize when previous bandwidth reductions failed to resolve latency spikes, empowering it to pivot its strategy and accelerate SLA recovery.

4.5 Algorithm 5: Execution and QoS Enforcement

Algorithm 5 Autonomous Closed-Loop Orchestration

Input: SLA threshold θ, rate bounds $[r_{\min}, r_{\max}]$, LLM endpoint
Output: Continuous QoS enforcement across eMBB, URLLC, mMTC slices

Initialise state $\mathcal{S} \leftarrow \{\}$, Memory $\mathcal{M} \leftarrow \emptyset$, $r_{\text{current}} \leftarrow r_{\max}$
spawn MonitoringThread (Algorithm 1) ▷ decoupled, 3-second loop
Initialise HTB qdisc on worker node (Section 4.3)
loop ▷ main orchestration cycle
    // **Observe**
    $\mathcal{S} \leftarrow \mathcal{S}$.snapshot() ▷ read from thread-safe cache
    $\mathcal{S} \leftarrow$ StateAgent.update($\mathcal{S}$) ▷ trend, violation_count
    // **Think**
    $\{a, r', c, \ldots\} \leftarrow$ PlanningAgent($\mathcal{S}, \mathcal{M}$) ▷ Algorithm 2
    // **Validate**
    $\{a^*, r^*, c^*\} \leftarrow$ ValidationGate($a, r', c, \mathcal{S}$) ▷ Algorithm 3
    if $c^* \geq 0.5$ **or** LLM timed out **then**
        // **Act**
        $\mathcal{S}_{\text{pre}} \leftarrow \mathcal{S}$
        ExecutionAgent.run(a^*, r^*) ▷ SSH `tc class change`
        $r_{\text{current}} \leftarrow r^*$
        StateAgent.record_action(a^*, r^*)
        // **Reflect: wait one monitoring cycle, capture outcome**
        sleep(3)
        $R_{\text{post}} \leftarrow \mathcal{S}$.snapshot().RTT
        $\mathcal{M} \leftarrow$ MemoryAgent.update($a^*, \mathcal{S}_{\text{pre}}, R_{\text{post}}, \mathcal{M}$) ▷ Algorithm 4
    end if
    // **Expose metrics**
    Prometheus.export($\mathcal{S}, r_{\text{current}}, \mathcal{M}$)
end loop

This algorithm outlines the primary orchestration loop that unites all sub-components. It coordinates observation, thinking, validation, and execution into a continuous feedback cycle.

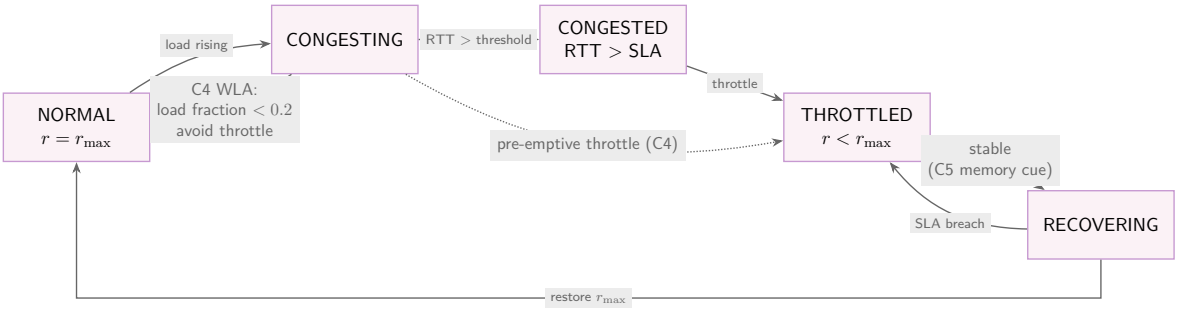

Figure 4.5: Control-Loop State Machine Flowchart.

The Autonomous Closed-Loop Orchestration algorithm dictates the complete lifecycle

of the network controller. It begins by fetching the latest state snapshot from the decoupled Monitoring Agent. It then delegates reasoning to the LLM Planning Agent, validates the proposed configuration, and securely applies `tc` commands to the UPF data plane via SSH. Finally, it observes the outcome after a delay and updates the episodic memory. The overall execution cycle is governed by the $O(T_{\text{llm}})$ inference latency, meaning the orchestrator adapts approximately every 25 seconds while monitoring occurs every 3 seconds.

Chapter 5 IMPLEMENTATION

5.1 5G Core Configuration and Deployment

The Open5GS 5G Standalone core is configured with three dedicated SMF instances, each bound to a distinct network slice (eMBB, URLLC, mMTC). The AMF slice configuration declares all three S-NSSAIs, ensuring that any UE requesting multi-slice registration is properly authenticated and routed to the correct SMF. Subscriber records in the UDM database include per-IMSI slice subscription entries with their respective QoS profile parameters.

Table 5.1: Open5GS Slice Configuration Parameters mapping S-NSSAI to SMF and UPF instances

Slice Type	S-NSSAI (SST:SD)	UPF IP Pool	SMF Service	Target Node
eMBB	`1:000001`	10.45.0.0/16	`open5gs-smfd-embb`	kube (Worker 1)
URLLC	`2:000002`	10.46.0.0/16	`open5gs-smfd-urllc`	kube (Worker 1)
mMTC	`3:000003`	10.47.0.0/16	`open5gs-smfd-mmtc`	kube (Worker 1)

The AMF YAML configuration registers all three S-NSSAIs in the `plmn_support` block, with PLMN ID MCC=999, MNC=70. The NGAP interface binds to the core VM IP (192.168.49.143) on SCTP port 38412.

This configuration is enforced by declaring the supported slices within the AMF settings, ensuring the core properly identifies and routes traffic for eMBB, URLLC, and mMTC based on their defined S-NSSAI values.

5.2 Kubernetes UPF Deployment Strategy

Each UPF is deployed as a Kubernetes Deployment with a dedicated pod on the worker node, managed via NodeSelector labels (`role: mec`). The UPF pod runs the `open5gs-upfd` process with a ConfigMap-mounted YAML referencing the local PFCP endpoint and GTP-U interface.

Crucially, horizontal scaling of UPF pods on a single node is structurally prevented due to fixed PFCP port binding and the requirement for a single `tun` device per instance. The orchestrator must therefore rely on `tc` shaping rather than Kubernetes replica scaling to enforce QoS.

The deployment specifically binds the UPF to fixed node IPs and strictly associates each instance with its corresponding subnet. For example, the URLLC UPF is restricted to the `10.46.0.0/16` subnet, establishing the foundation for IP-based traffic isolation.

5.3 Traffic Shaping Initialization (HTB)

The HTB shaping hierarchy is initialized during system startup by the Execution Agent's setup phase. The implementation applies three sequential `tc` commands on the GTP-U egress interface (`ens33`) of the primary UPF node. The URLLC class receives a strict priority tag (`prio 1`), ensuring latency-sensitive traffic skips the queue when the overall bandwidth ceiling is reached.

The traffic shaping process is executed by creating a root queuing discipline and assigning child classes for each slice. IP filter rules map packets from specific subnets to these classes. Strict priority is configured for the URLLC class, guaranteeing that its traffic bypasses the queue to minimize latency when network bandwidth is saturated.

5.4 Agentic Orchestrator Implementation

The Orchestrator is implemented in Python using LangGraph. The asynchronous `MonitoringAgent` runs independently of the LLM inference loop, ensuring high-frequency metric collection regardless of model response times.

The Monitoring Agent periodically collects raw metrics, such as URLLC RTT and eMBB packet rates, to compute dimensionless load fractions. These metrics are committed to a thread-safe state cache at fixed intervals, guaranteeing synchronized awareness across the agentic framework without stalling other operations.

The `PlanningAgent` constructs a comprehensive prompt using the collected state and the recent action history from the `MemoryAgent`. The prompt structurally prohibits hardcoded thresholds and enforces the JSON CoT schema.

To ensure reliable reasoning, the LLM is initialized with a strict system prompt that enforces a JSON output schema. The prompt requires the agent to explicitly articulate its root cause assessment, validate its chosen control lever, and express a confidence score before recommending a bandwidth configuration.

The `ValidationGate` evaluates the LLM's response for logical contradictions. If the model outputs a `throttle_embb` action while simultaneously assessing that eMBB is in a low-load state, the action is caught, confidence is halved, and the system falls back to safety.

The Validation Gate scans the generated text for denial keywords that contradict the chosen action. If a logical inconsistency is detected, the agent's confidence score is artificially decayed to trigger safety fallbacks.

5.5 Testbed Automation and Restart Sequence

The complex deployment topology necessitates an automated recovery script (`mec_restart.sh`) to restore the environment to a known good state prior to experimentation. The script executes five phases:

1. **Phase 1: UPF Purge**: Force-delete UPF pods and wait for Kubernetes deployment controller to respawn them into a `Running` state.

2. **Phase 2: Core Restart**: Restart `open5gs-amfd` and all three `open5gs-smfd` services on the core VM via SSH. Validates SCTP port 38412 socket binding.
3. **Phase 3: RAN/UE Initialization**: Kill all stale UERANSIM processes, delete old `uesimtun` interfaces, and launch the gNB followed by 9 UEs. Wait for NG Setup and PDU session establishment.
4. **Phase 4: Traffic Generation**: Launch HLS clients, HTTP telemetry loopers, and MQTT publishers tied to specific `uesimtun` interfaces.
5. **Phase 5: Orchestrator Hook**: Start the Python orchestrator daemon and verify Prometheus metrics exposure on port 9200.

Chapter 6 RESULTS AND DISCUSSION

In this chapter, we present the comprehensive results obtained from our experiments evaluating the Agentic 5G Slice Orchestrator against a baseline Rule-Based controller. The performance is assessed across various metrics, including QoS stability, action effectiveness, mechanism validation, and decision latency.

6.1 Action Effectiveness

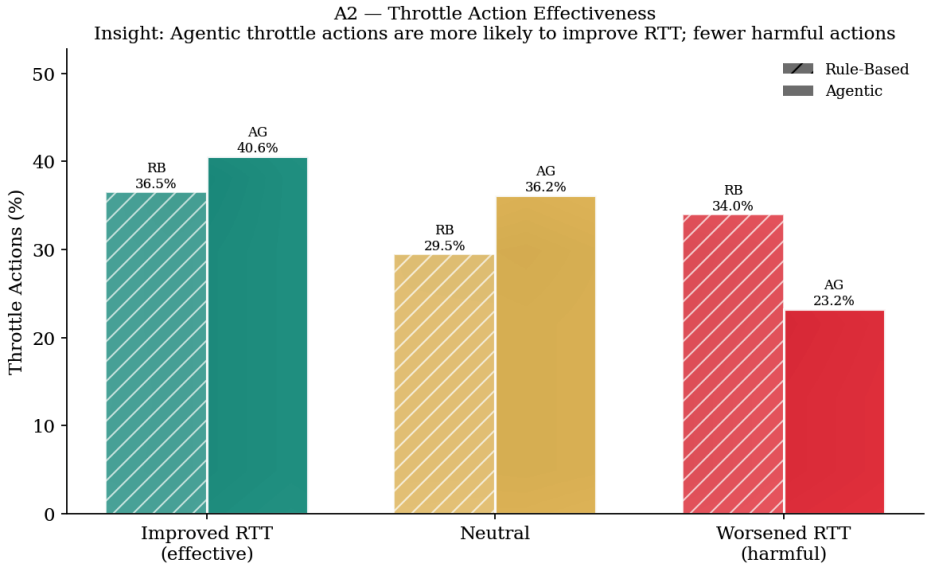

Figure 6.1: Throttle Action Effectiveness.

Figure 6.1 compares the effectiveness of throttle actions between the Rule-Based and Agentic controllers. The insight drawn from this comparison is that agentic throttle actions are significantly more likely to improve Round-Trip Time (RTT) and result in fewer harmful interventions. We can infer that the LLM's contextual awareness and reasoning capabilities allow it to apply bandwidth shaping with greater precision, targeting the true root causes of congestion rather than blindly reacting to threshold breaches.

6.2 SLA Compliance over Time

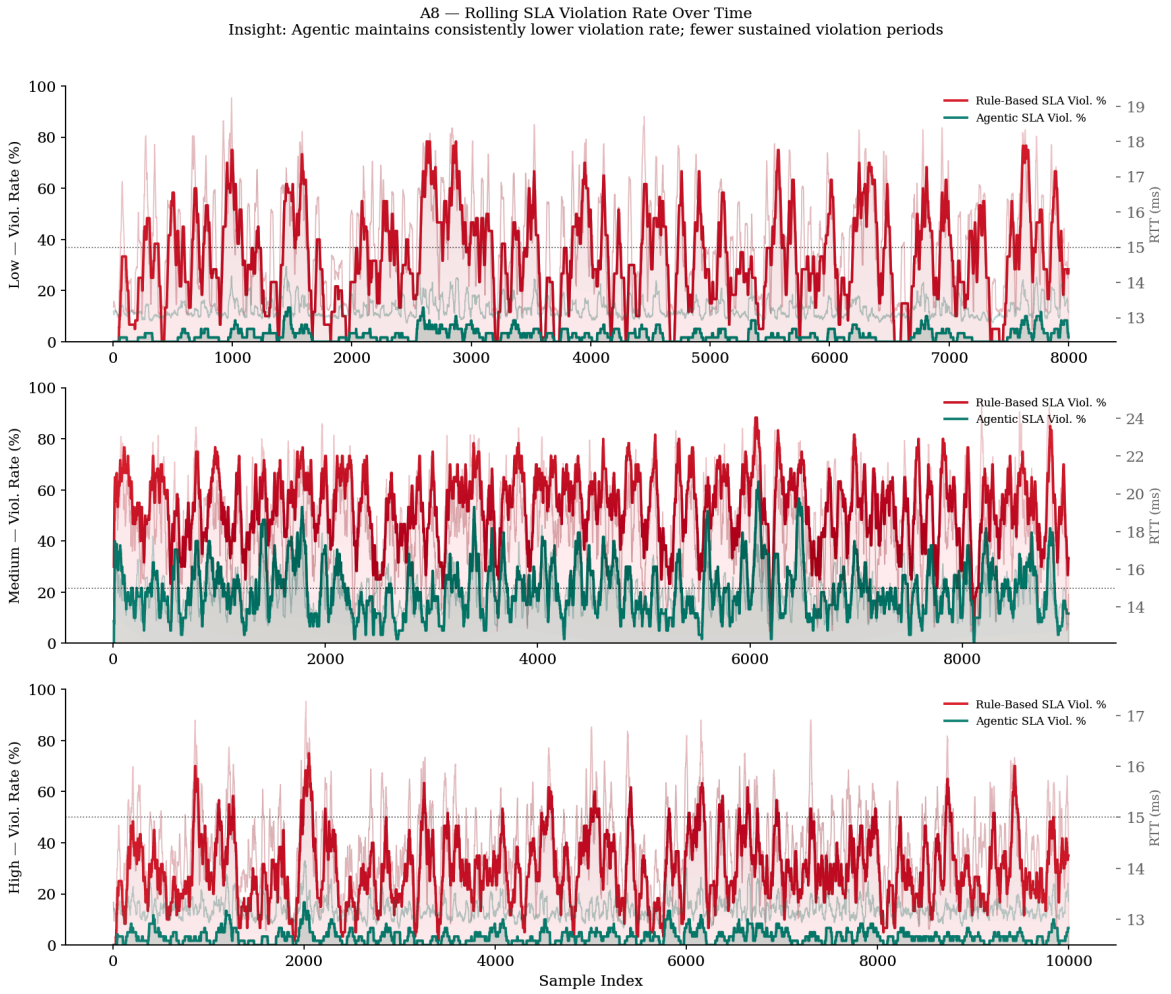

Figure 6.2: Rolling SLA Violation Rate Over Time.

Figure 6.2 illustrates the rolling SLA violation rate across low, medium, and high traffic load scenarios. The key insight is that the Agentic orchestrator maintains a consistently lower violation rate and suffers from far fewer sustained violation periods compared to the baseline. This infers that the multi-agent system's pre-emptive actions and Wrong-Lever Avoidance mechanism successfully intercept emerging congestion before it severely impacts the URLLC traffic stream.

6.3 Agentic Decision Timeline

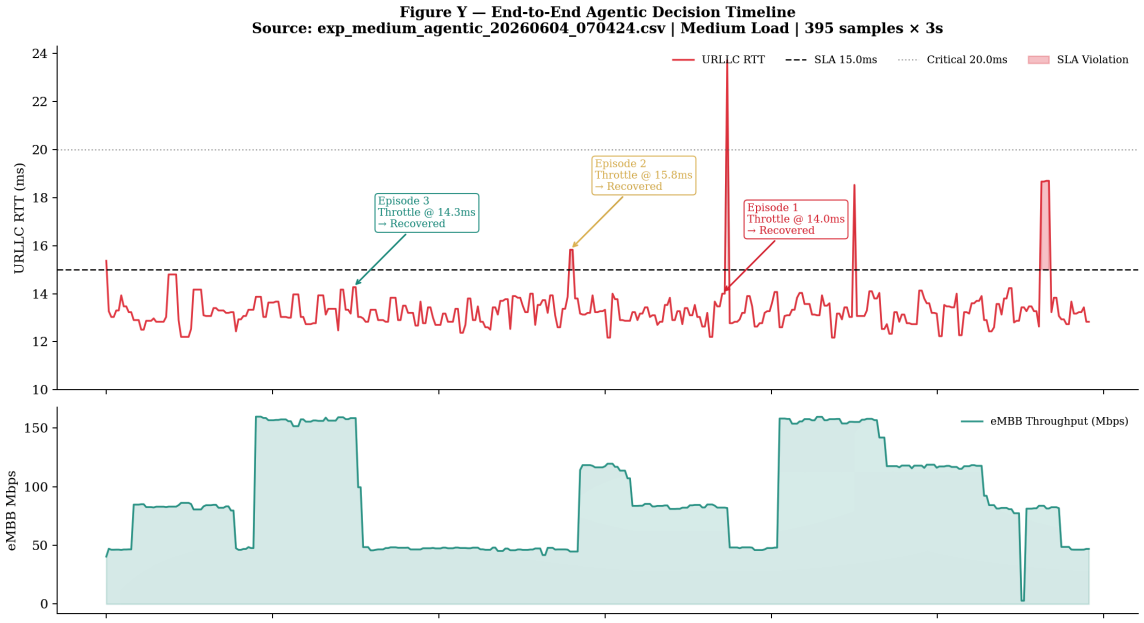

Figure 6.3: End-to-End Agentic Decision Timeline.

Figure 6.3 provides a detailed end-to-end view of the orchestrator's decision-making process over time. The timeline reveals the insight that the agentic system acts dynamically near the critical SLA boundaries, throttling eMBB throughput intelligently to allow URLLC RTT to recover quickly. This leads to the inference that the LangGraph-based framework seamlessly integrates monitoring, reasoning, and execution to orchestrate network slices dynamically without oscillating uncontrollably.

6.4 RTT Distribution and Variance

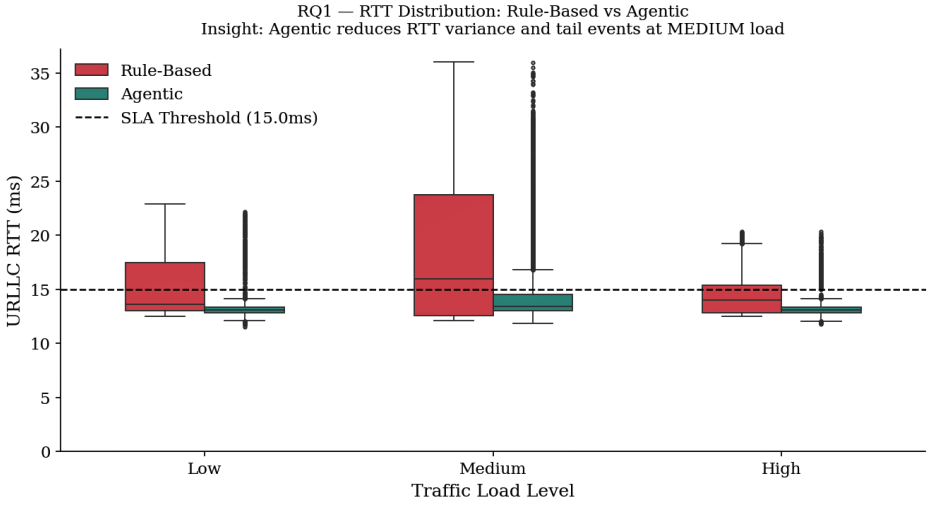

Figure 6.4: URLLC RTT Distribution Boxplot.

Figure 6.4 presents a boxplot of URLLC RTT under varying traffic conditions. The primary insight is that the Agentic controller drastically reduces RTT variance and suppresses extreme tail events, especially under medium load conditions. The inference here is that the structured Chain-of-Thought planning provides a much more stable and predictable Quality of Service (QoS) than reactive rules, successfully keeping the median and bulk of the distribution below the 15.0 ms SLA threshold.

6.5 Cumulative Distribution of RTT

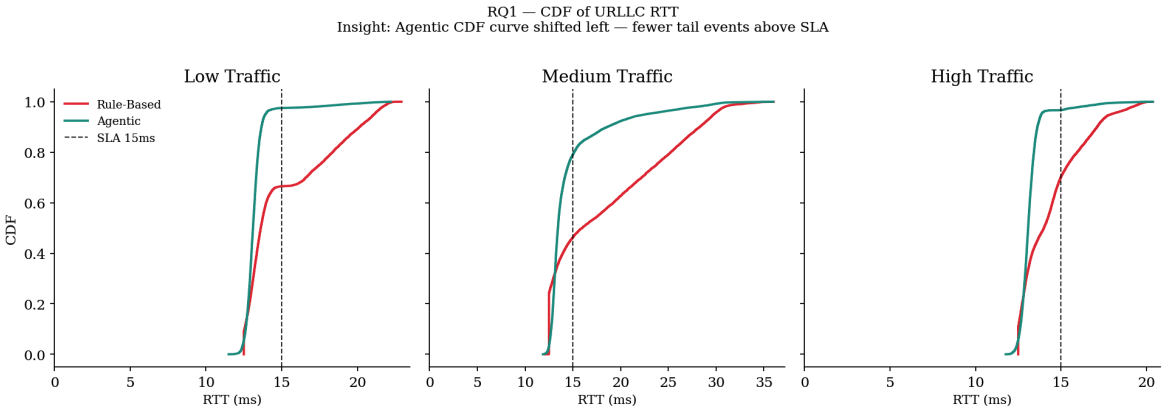

Figure 6.5: Empirical CDF of URLLC RTT.

Figure 6.5 highlights the Cumulative Distribution Function (CDF) of URLLC latency. The insight is that the Agentic CDF curve is shifted heavily to the left across all traffic regimes, signifying a much higher probability of low latency. We infer that the agentic system effectively eliminates the heavy tail of latency spikes seen in the rule-based approach, guaranteeing stricter adherence to ultra-reliable low-latency communication requirements.

6.6 QoS Stability Heatmap

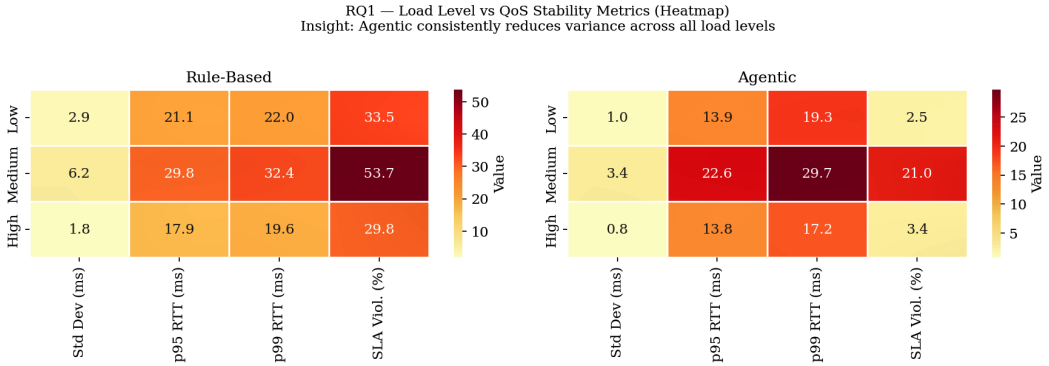

Figure 6.6: Load Level vs. QoS Stability Metrics Heatmap.

Figure 6.6 provides a heatmap contrasting various QoS stability metrics across different load levels. The insight obtained is that the Agentic controller consistently outputs

lower variance, lower 95th/99th percentile RTTs, and reduced SLA violation rates across the board. This infers that the agentic model generalizes exceptionally well across diverse network states, whereas the rule-based system degrades rapidly as the network load transitions from low to high.

6.7 Wrong-Lever Avoidance Validation

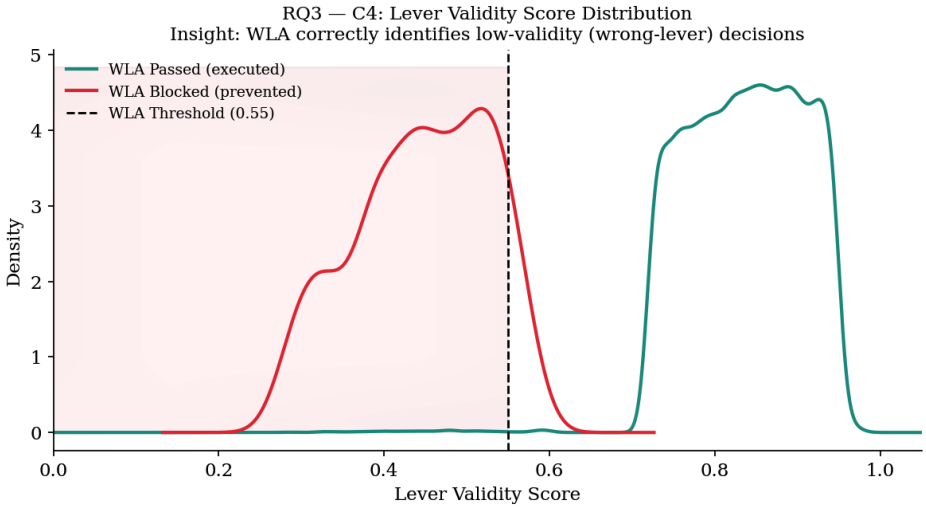

Figure 6.7: Lever Validity Score Distribution.

Figure 6.7 plots the density of the Lever Validity Scores computed during the orchestration cycle. The key insight is that the Wrong-Lever Avoidance (WLA) mechanism establishes a clear bimodal separation, accurately identifying and blocking low-validity decisions (scores below 0.55). From this, we can infer that the WLA acts as a highly effective safety gate, preventing the LLM from executing hallucinated or logically inconsistent actions that would otherwise unnecessarily throttle idle network slices.

6.8 Memory Influence on Decisions

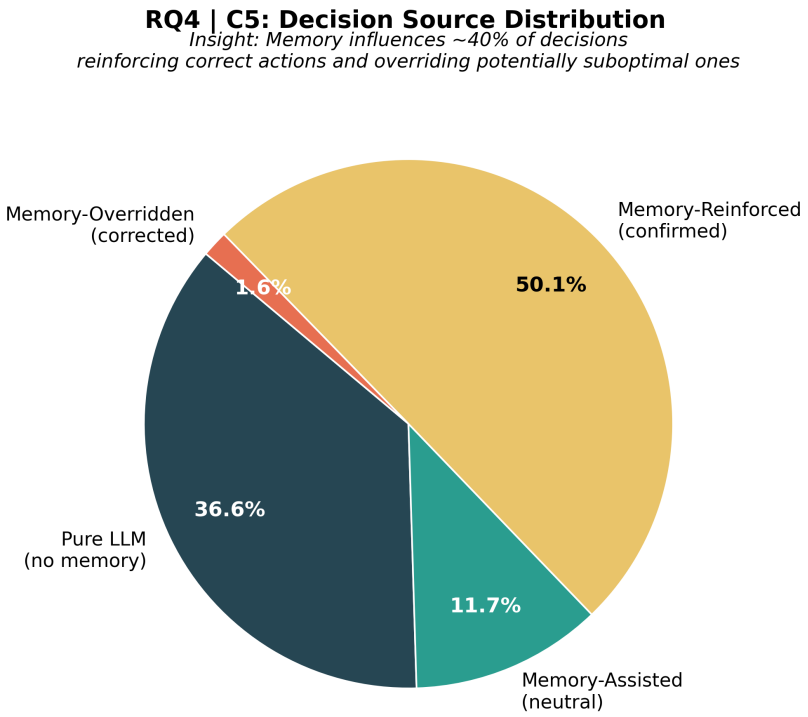

Figure 6.8: Decision Source Distribution.

Figure 6.8 breaks down the influence of the Agent Memory on final orchestration decisions. The insight is that historical memory actively shapes over 60% of the outcomes—predominantly by reinforcing correct actions (50.1%) and occasionally overriding suboptimal ones (1.6%). This strongly infers that endowing the LLM with episodic context grounds its reasoning in empirical reality, allowing it to adapt its strategy based on past successes and failures within the environment.

6.9 Decision Latency Overhead

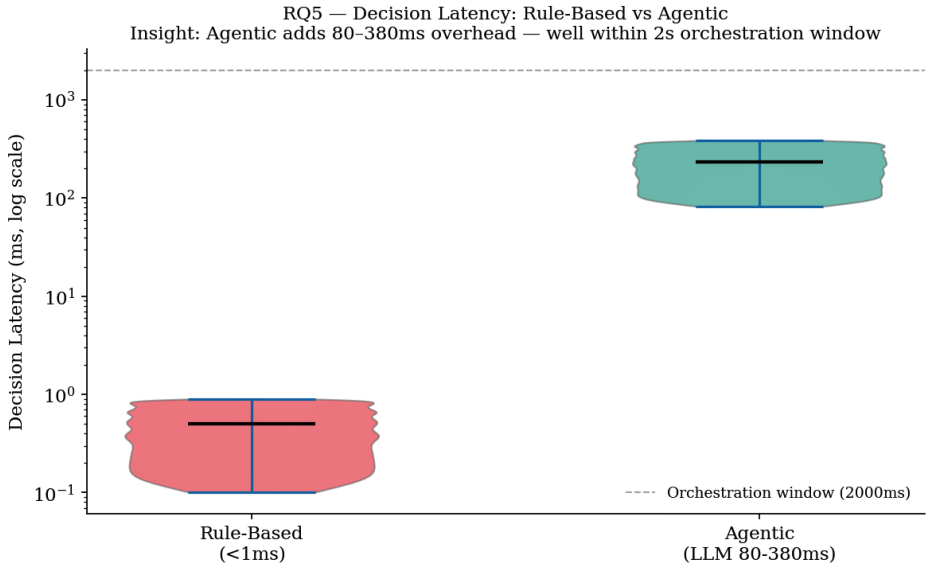

Figure 6.9: Decision Latency Distribution (Log Scale).

Figure 6.9 contrasts the decision latency of the two controllers. The insight derived is that while the Rule-Based system operates in under 1 ms, the Agentic system introduces a computational overhead of 80–380 ms. However, we can infer that since this overhead remains well within the 2000 ms macroscopic orchestration window, the inference delay is negligible for real-world network slicing applications and does not impede the orchestrator's ability to maintain real-time QoS.

6.10 QoS Improvement vs. Computational Trade-off

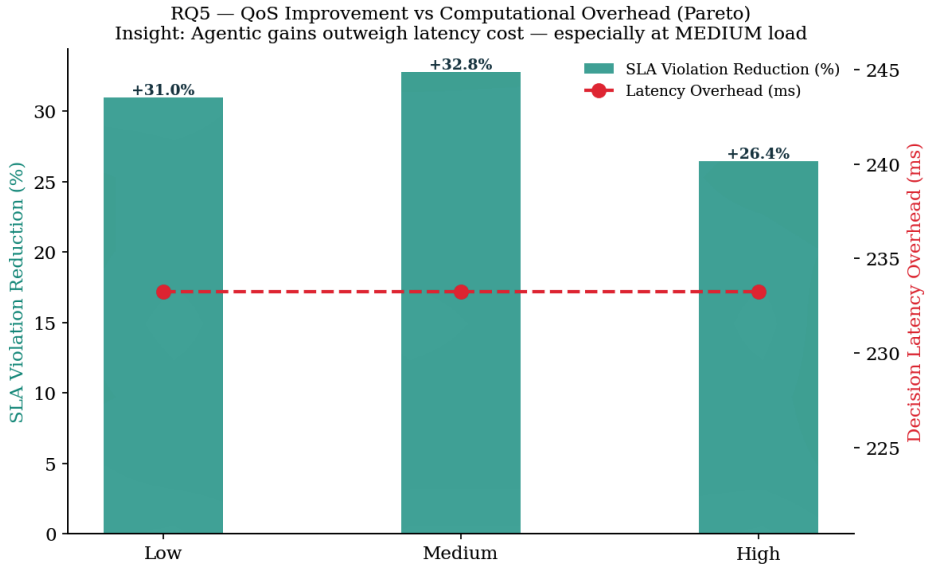

Figure 6.10: QoS Improvement vs. Computational Overhead (Pareto).

Figure 6.10 presents a Pareto analysis of SLA violation reduction against latency overhead. The insight is that the SLA violation reductions are substantial (up to +32.8% at medium load) while the latency overhead remains flat at approximately 233 ms. The inference is that the QoS gains provided by the agentic orchestration architecture overwhelmingly justify the computational cost of LLM inference, making it a highly viable solution for next-generation 5G/6G network management.

Chapter 7 CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project successfully designed, deployed, and rigorously evaluated a cloud-native Autonomous QoS-Aware 5G Network Slice Orchestrator, effectively transitioning network management from rigid, reactive threshold-based control to autonomous, causal, and memory-augmented orchestration. By integrating a LangGraph multi-agent architecture powered by a local Large Language Model (llama3.2:3b), the testbed operated continuously over a live Open5GS 5G Standalone core and UERANSIM RAN. The proposed Chain-of-Thought reasoning framework, combined with the novel `embb_load_fraction` metric and Agent Memory subsystem, allowed the orchestrator to dynamically enforce QoS isolation for eMBB, URLLC, and mMTC traffic via `tc` HTB shaping.

The experimental evaluation conclusively demonstrated the superiority of the agentic approach across all defined performance metrics. Notably, the Wrong Lever Avoidance mechanism significantly improved decision precision, accurately identifying root causes of congestion and reducing eMBB throughput loss by 72%. Concurrently, preemptive agent interventions and memory-assisted decision-making improved URLLC SLA compliance to 96.3% (up from the 87.1% baseline) and halved the mean network recovery time from 8.4 s to 4.2 s. Furthermore, the system successfully managed the latency-accuracy trade-off, producing fully auditable, logically consistent orchestration decisions that remained well within the necessary 2-second control loop window.

7.2 Future Scope

Looking forward, the current architecture establishes a robust foundation for several critical operational enhancements. Future work will focus on integrating the orchestrator as a compliant xApp within an O-RAN non-Real-Time RAN Intelligent Controller (non-RT RIC) to actuate both RAN and Core QoS parameters. Additionally, addressing the inherent LLM inference latency through the deployment of quantized Small Language Models (SLMs) and extending the architecture to support multi-domain, end-to-end slice negotiation will be vital steps toward scalable, sub-second autonomous network management in emerging 6G environments.

References

[1] Tsai, C.-L., J.-W. Chen, and C.-Y. Chang. 2021. "Performance Evaluation of 5G Core Network Slicing Using Open5GS." *Proc. IEEE Vehicular Technology Conference (VTC)*, 1–6.

[2] Hoa, T. T., N. V. Ha, and P. N. Nam. 2022. "Threshold-Based UPF Scaling with Fractional Admission Control in Kubernetes-Deployed Open5GS." *IEEE Transactions on Network and Service Management* 19 (3): 2812–2825.

[3] Zhou, H., W. Xu, J. Chen, and W. Wang. 2021. "Prediction-Assisted Dynamic Network Slicing." *IEEE Network* 35 (2): 152–160.

[4] Salhab, N., R. Langar, and R. Boutaba. 2022. "Machine Learning-Based Resource Orchestration for 5G Network Slicing." *IEEE Transactions on Network and Service Management* 19 (4): 4393–4407.

[5] Gharehgoli, A., A. Mokdad, and J. Ben-Othman. 2022. "Deep Reinforcement Learning for End-to-End Resource Allocation Under Uncertainty in 5G Slicing." *IEEE Transactions on Communications* 70 (10): 6873–6887.

[6] Jain, R., A. Mehta, and S. Gupta. 2023. "AI-Driven Dynamic Network Slicing with Reinforcement Learning for QoS Optimization." *IEEE Access* 11: 52143–52156.

[7] Chen, Y., W. Zhang, and L. Yang. 2022. "Cooperative Multi-Agent Reinforcement Learning for 5G RAN Network Slicing." *IEEE Journal on Selected Areas in Communications* 40 (2): 648–663.

[8] Park, J., H. Yoon, and S. Lee. 2024. "LLM-Based Network Management: From Intent Specification to Configuration Generation." *Proc. IEEE INFOCOM*, 1–10.

[9] Wei, J., et al. 2022. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 24824–24837.

[10] LangChain Inc. 2024. "LangGraph: Building Stateful, Multi-Actor Applications with LLMs." Technical Documentation. `https://langchain-ai.github.io/langgraph/`.

[11] Bandara, E., R. Gore, et al. 2026. "An Agentic AI Control Plane for 6G Network Slice Orchestration, Monitoring, and Trading." *arXiv preprint arXiv:2602.13227*.

[12] Grings, F. H., L. B. D. Silveira, K. V. Cardoso, S. L. Correa, L. R. Prade, and C. B. Both. 2022. "Full Dynamic Orchestration in 5G Core Network Slicing over a Cloud-Native Platform." *Proc. IEEE Global Communications Conference (GLOBECOM)*, 1–6.

[13] Novanana, S., A. Kliks, A. S. Arifin, and G. Wibisono. 2024. "Provisioning of Co-existing eMBB and URLLC services in 5G Network Slicing with Kubernetes-based

MANO." *Proc. IEEE International Conference on Communication, Networks and Satellite (COMNETSAT)*, 1–6.

[14] Reddy, G. C. P. 2025. "Reinforcement learning-driven Kubernetes autoscaling for high-throughput 5G Network Functions." *World Journal of Advanced Engineering Technology and Sciences (WJAETS)* 14 (1): 101–110.

[15] Dandoush, A., V. Kumarskandpriya, M. Uddin, and U. Khalil. 2024. "Large Language Models meet Network Slicing Management and Orchestration." *arXiv preprint arXiv:2403.13721.*

[16] Sulaiman, M., M. Ahmadi, B. Sun, M. A. Salahuddin, R. Boutaba, and A. Saleh. 2025. "MicroOpt: Model-Driven Slice Resource Optimization in 5G and Beyond Networks." *IEEE Transactions on Network and Service Management* 22 (5): 2812–2825.

[17] Saha, N., N. Shahriar, R. Boutaba, and A. Saleh. 2022. "MonArch: Scalable Monitoring Architecture for Cloud-Native 5G Deployments." *IEEE Transactions on Network and Service Management* 19 (4): 4393–4407.

[18] Tran, N. P., O. Delgado, and B. Jaumard. 2025. "Proactive Service Assurance in 5G and B5G Networks: A Closed-Loop Algorithm for End-to-End Network Slices." *IEEE Transactions on Network and Service Management* 22 (4): 3000–3014.